

Autonomous Agentic RAG with Distributed Vector Databases for Scalable Information Retrieval

Tural Mehtiyev Anirudh Kaluri Sagar Dama
North Carolina State University

ABSTRACT

This project presents a distributed system architecture that integrates two key components: Agentic Retrieval-Augmented Generation (RAG) with autonomous agents operating on separate nodes, and a distributed vector database for efficient similarity search and retrieval. The proposed system aims to address the scalability challenges in both RAG implementations and vector search by creating a unified distributed architecture. Our approach synthesizes distributed systems principles with recent advances in vector databases and autonomous AI agents to create a more robust and scalable information retrieval and generation system.

KEYWORDS

Distributed Systems, Retrieval-Augmented Generation, Autonomous Agents, Vector Databases

1 INTRODUCTION

Distributed Systems are designed to handle large work loads and provide failure-proof computing. They provide computing power across multiple nodes giving users higher availability, reliability, and elasticity. At the Data Layer, the concept of distribution is data being spread across nodes. This distribution is transparent to the client using the data services. The concept of transaction should be revisited in such a scenario. For example, for a table that is distributed across instances, finding the correct node to execute a query requires finding the node with relevant data first. For a distributed query that requires data from multiple tables spread across multiple nodes, the query needs to shortlist the nodes, scan relevant data and aggregate it before sending it as an output.

Recent breakthroughs in Large Language Models (LLMs) led to an increase in the use case of Vector Databases. LLMs can read, understand, and respond to natural languages like English. The underlying technology for LLMs are deep learning models. These models are trained on data to teach the natural language abilities. However, the validity of the data on which the LLMs are trained expire with time. This requires LLMs to be trained on the new updated data to replace the older versions. However, re-training the deep learning model is an expensive operation. Moreover, proprietary models are closed to be trained by retail or enterprise users. On top of that, the training of LLMs may be imbalanced with less knowledge in a particular domain. All these drawbacks led to the use of Retrieval Augmented Generation technique (RAG).

RAG technology leverages data store to provide additional context to the LLMs. That is, LLMs can use new data without being trained on it. The new data helps LLMs specialize in a particular domain. For example, a data store with 10000 documents from the field of medicine allows the LLM to answer user queries about healthcare more accurately equipped with latest information. Vector Databases (vector dbs) are the data stores that allow for these

specialized data operations. In a vector database, natural languages text/documents are stored in the form of vectors a.k.a embeddings.

With LLMs being incorporated across the software product space for business solutions, using vector database is a necessity. However, companies that traditionally use relational databases at scale cannot shift to products that exclusively support vector operations. Hence, PostgreSQL came up with an extension called pgVector allowing embedding data type and related operations. However, in larger workloads, high performance CRUD operations require multiple nodes running database servers. In other words, the embeddings data is spread across nodes in a distributed environment. However, databases like Postgres by default doesn't support distributed behaviour. In this project, we propose to implement a distributed behavior for a vector database.

This project proposes a dual-focused distributed architecture that addresses both distributed systems and RAG challenges by:

- Taking a database without distributed capabilities and implementing a distributed behavior for vector database for scalable similarity search.
- Deploying scalable autonomous RAG agent

2 PROPOSED APPROACHES

Our proposed system architecture consists of two main components that work together to create a scalable and efficient distributed RAG system. We aim to provide distributed functionality to a database, specifically, in the context of sharding vector data. This can be done at two levels: database and multi-agent RAG. We detail each component below.

2.1 Distributed Vector Database

At the database level we aim to implement a Coordinator-Worker architecture. The coordinator takes the role of a master. The Master node stores all the metadata of the workers. Each worker node has access to one or more shards. The master node is responsible for mapping new queries and data to relevant worker node. The worker node will perform the read and write operations on the specific piece of data with which it interacts.

The detailed implementation can be undertaken with different previously tested approaches: 1) In Citus, a distributed extension to postgres, the Coordinator-worker implementation is straightforward with a provision of User Defined Functions to perform operations over the metadata in the coordinator node. 2) In Milvus, a vector database, a single writer node and multiple reader nodes have been populated. However, the reader nodes are stateless compute instances. The data is stored in a shared common storage (Amazon S3). The Coordinator manages the write node and other reader nodes with each reader node associated with a logical shard. 3) In SingleStore, two types of nodes exist- aggregators that communicate with clients and leaves which hold partitions of data. The

coordinator uses hashing over a shard key to determine the node on which read and write operations are to be performed.

However, having a master node is a bottleneck for high performance CRUD operation. In Milvus, a flat hierarchical approach was proposed where DML (Data Manipulation Language) queries were delegated to the worker nodes while using the Coordinator role for DDL(Data Definition Language) operations as they are fewer.

2.1.1 Datasets. The primary application of the distributed vector database we are building will be in enabling RAG systems. In order to test the working of the distributed vector db's functionality, we will be utilizing testing data over which inserts, deletes and updates will be performed. The data is purely text based documents in English language. The data will be collected from different sources across the Internet such as Wikipedia dumps, PubMed Articles, HuggingFace datasets etc.. As we will be testing sharding for our vector database, data related to different domains/topics like education, healthcare, sports etc will be collected for diversity in the data.

2.2 Distributed Agent Architecture

The distributed agent architecture proposed in this project implements a multi-node system where specialized agents operate autonomously while maintaining coordinated behavior. Each agent type is designed to perform a specific function within the Retrieval-Augmented Generation (RAG) pipeline. The following agents represent the initial list of agents we are planning to implement, with the flexibility to extend or refine them as the system evolves.

2.2.1 Query Analysis Agents. Query Analysis Agents serve as the system's front-end processors. They employ natural language understanding (NLU) techniques to decompose complex user queries into manageable subtasks. This hierarchical task decomposition framework optimizes the system for parallel processing and efficient resource allocation.

2.2.2 Vector Search Agents. Vector Search Agents manage distributed vector search operations. These agents implement query planning and optimization strategies, coordinating with the distributed vector database to execute similarity searches.

2.2.3 Retrieval Agents. Retrieval Agents handle document retrieval based on vector search results. They prioritize retrieval operations using caching strategies and adapt dynamically to query requirements and system load.

2.2.4 Synthesis Agents. Synthesis Agents combine retrieved information using language generation techniques to ensure coherent and contextually appropriate responses. These agents employ adaptive generation strategies that balance response quality with system resource utilization.

2.2.5 Agent Coordination. The coordination between these agents is facilitated by an orchestrator agent that dynamically allocates tasks based on real-time performance metrics and workload distribution. This approach aligns with principles of distributed multi-agent systems, where agents collaborate asynchronously to achieve shared goals [9].

3 RELATED WORK

3.1 Distributed Database

The review study [11] provides overview of vector databases. It explains in detail the flow of work for vector databases-from indexing to querying-and the main techniques to improve the search efficiency by applying various similarity search algorithms such as K-Means, LSH, HNSW, PQ, among others, along with similarity metrics: Euclidean Distance, Dot Product, and Cosine Similarity. Results from the survey also compared widely used vector database products, such as Pinecone, Chroma, and Milvus, with respect to features and applications. Lastly, the paper discusses the direction of future research, mainly on improving the algorithms for searching and scalability while increasing integration with LLMs for better retrieval over high-dimensional and large-scale datasets.

The study [3] propose different approaches for similarity search by providing efficient algorithms. The paper focusses on overcoming computational performance issues like memory inefficiencies. It also provides a github repository link with its implementation. The paper provides improvement over brute-force search and approximate nearest neighbour search. The result is that they achieve 8.5x faster performance than previous GPU implementations of the algorithms. The approaches are then integrated into the Faiss library for open source access.

The study following study [4] provides a review of vector database management systems (VDBMSs). Traditional databases have issues with handling vector data as similarity computations are expensive. Also there is a lack of indexable structures and support for hybrid queries. This study emphasizes on scalability and sharding techniques where the partitioning of vectors happen across multiple nodes. It discusses algorithms like HNSW which is graph based and KD-trees which is tree based. Optimization techniques including hardware accelerations via SIMD and GPUs are discussed in the study which are relevant to this project.

Vexless [7] is a vector database system developed using cloud functions for serverless computing. The paper throws light on distributed sharding, minimizing communication overhead, and reducing cold-start latency. Vexless utilizes a global orchestrator to dynamically manage workloads to cloud function instances based on available resources, ensuring efficient resource management. To remove any communication bottlenecks, it makes use of stateful cloud functions and an optimized messaging mechanism using Azure Durable Functions and Queue Storage, removing the need for costly synchronization. Vexless also provides workload-aware function lifetime management. This helps in mitigating delayed starts. The study provides experimental results that shows that Vexless achieves cost efficiency for bursty workloads. All this while maintaining query performance comparable to traditional VM-based vector databases.

SingleStore-V [1] is a distributed database that integrates vector search with SQL compatibility. It employs horizontal sharding, where data is partitioned across leaf nodes based on a shard key. This ensures uniform distribution. Data is partitioned based on hashing and stored in an in-memory rowstore. The data is further divided into segments with each segment having its own vector index. Aggregator nodes coordinate queries, distributing execution to leaf nodes, which scan, filter, and merge results before returning

final query outputs. This design ensures high availability, scalability, and efficient hybrid query execution combining vector and non-vector data in a fully distributed environment.

Citus [2] created a Coordinator-Worker structure for distributed workloads. Citus leveraged the extension facility of Postgres to create a sharding layer or interface. This allowed the creation of a table across multiple nodes. For distributed queries, Citus provided a custom query planner and executor to search and aggregate the data. In high CRUD workloads, however, Citus provides a falter hierarchy by delegating the tasks of handling metadata of the distributed database across the worker nodes. Sharding is done based on hashing the distributed column. Citus also comes with a Shard rebalancer to make sure of uniform distribution across the nodes.

Milvus [8], employs a shared-storage architecture, where all vector data is stored centrally in Amazon S3 or HDFS. This design ensures data consistency. The compute layer doesn't have a state. This means that reader instances don't have data locally. However, they have caching abilities. Each reader is assigned a logical shard using consistent hashing. The coordinator dynamically routes queries to the appropriate reader, ensuring load balancing and automatic failover in case of instance failure. Kubernetes manages scaling, allowing new reader instances to be added or reassigned dynamically without redistributing physical data. The system supports multi-node deployments, heterogeneous computing (CPU+GPU), and advanced query processing such as attribute filtering and multi-vector queries.

3.2 Distributed Agent

Recent advancements in multi-agent frameworks for Retrieval-Augmented Generation (RAG) systems have demonstrated potential in enhancing the efficiency and accuracy of complex tasks.

Wu et al. [10] introduced **AutoGen**, a multi-agent framework designed to enable autonomous collaboration among agents for complex tasks. AutoGen allows developers to build applications by integrating multiple conversable agents that interact with each other, leveraging large language models (LLMs), tools, and human inputs. The framework supports hierarchical agent structures where specialized agents handle tasks such as query analysis, information retrieval, and response synthesis. Experiments demonstrated that AutoGen significantly improves response accuracy and completeness compared to single-agent approaches.

A research by Liu et al. [5] explores application of multi-agent RAG architecture in time series analysis. The framework employs a hierarchical structure where a master agent orchestrates sub-agents specialized in specific tasks. This approach achieves state-of-the-art performance across major time series benchmarks by addressing challenges like spatio-temporal dependencies and distribution shifts.

Finally, the survey by Singh et al. [6] explores the role of multi-agent collaboration in Agentic RAG systems. It emphasizes how multiple agents can dynamically divide tasks such as retrieval, reasoning, and synthesis to handle large-scale and distributed problems efficiently. The survey provides an overview of Agentic RAG, covering its core principles, the evolution of RAG frameworks, and a classification of different architectures. It also discusses key applications in fields like healthcare, finance, and education, along

with practical implementation strategies. Finally, it highlights challenges in scaling these systems and optimizing performance while offering insights into tools and frameworks for building Agentic RAG solutions.

Unlike previous studies, our work approaches Agentic RAG from an AI observability perspective. We focus on understanding and diagnosing failures within the RAG pipeline. We aim to perform root cause analysis by monitoring agent interactions, identifying bottlenecks in retrieval, reasoning, and synthesis stages, and detecting sources incorrect responses. By integrating observability techniques, we seek to improve system reliability and transparency, which will provide deeper insights into how multi-agent collaboration impacts the overall robustness of RAG-based architectures.

REFERENCES

- [1] Cheng Chen, Chenzhe Jin, Yunan Zhang, Sasha Podolsky, Chun Wu, Szu-Po Wang, Eric Hanson, Zhou Sun, Robert Walzer, and Jianguo Wang. 2024. SingleStore-V: An Integrated Vector Database System in SingleStore. *Proc. VLDB Endow.* 17, 12 (2024), 3772–3785. <https://doi.org/10.14778/3685800.3685805>
- [2] Umur Cubukcu, Ozgun Erdogan, Sumedh Pathak, Sudhakar Sannakkayala, and Marco Slot. 2021. Citus: Distributed PostgreSQL for Data-Intensive Applications. In *Proceedings of the 2021 International Conference on Management of Data (Virtual Event, China) (SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 2490–2502. <https://doi.org/10.1145/3448016.3457551>
- [3] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547. <https://doi.org/10.1109/TBDATA.2019.2921572>
- [4] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Vector Database Management Techniques and Systems. In *Companion of the 2024 International Conference on Management of Data (Santiago AA, Chile) (SIGMOD/PODS '24)*. Association for Computing Machinery, New York, NY, USA, 597–604. <https://doi.org/10.1145/3626246.3654691>
- [5] Chidakh Ravuru, Sagar Srinivas Sakhinana, and Venkataramana Runkana. 2024. Agentic Retrieval-Augmented Generation for Time Series Analysis. *arXiv preprint arXiv:2408.14484v1* (2024). <http://arxiv.org/abs/2408.14484v1>
- [6] Aditi Singh, Abul Ehtesham, Saket Kumar, and Tala Talaei Khoei. 2025. Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG. *arXiv:2501.09136 [cs.AI]* <https://arxiv.org/abs/2501.09136>
- [7] Yongye Su, Yinqi Sun, Minjia Zhang, and Jianguo Wang. 2024. Vexless: A Serverless Vector Data Management System Using Cloud Functions. 2, 3, Article 187 (May 2024), 26 pages. <https://doi.org/10.1145/3654990>
- [8] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data (Virtual Event, China) (SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 2614–2627. <https://doi.org/10.1145/3448016.3457550>
- [9] Michael Wooldridge. 2009. *An Introduction to Multi-Agent Systems*. Wiley Publishing.
- [10] Q. Wu, G. Bansal, J. Zhang, et al. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. <https://www.microsoft.com/en-us/research/publication/autogen-enabling-next-gen-llm-applications-via-multi-agent-conversation-framework/>
- [11] Xingrui Xie, Han Liu, Wenzhe Hou, and Hongbin Huang. 2023. A Brief Survey of Vector Databases. In *2023 9th International Conference on Big Data and Information Analytics (BigDIA)*. 364–371. <https://doi.org/10.1109/BigDIA60676.2023.10429609>